How EduVers Works

This guide explains EduVers in simple words, so anyone can understand it, even a Grade 5 student. Read it from top to bottom once. Then use it to get ready for your defense.

---

1. What is EduVers?

EduVers is a **school website**. Teachers put lessons and quizzes on it. Students read the lessons, take the quizzes and see their grades.

It is like a classroom, but on the internet.

---

2. Who uses it?

There are three kinds of people. Each one sees a different part of the website.

Person	What they do
Admin (school staff)	Makes the accounts, the sections, the subjects and the school year. Says "yes" or "no" when someone asks to join a class.
Teacher	Posts lessons, makes quizzes, sees the grades of their students.
Student	Reads lessons, takes quizzes, sees their own grades.

A student **cannot** open the teacher's pages. A teacher **cannot** open the admin's pages. The website checks this every single time.

---

3. The big idea: EduVers is a restaurant

When you click something on a website, two things happen:

1. You **ask** for something.
2. Something **comes back** to you.

Inside EduVers, helpers do the work in between. Think of a restaurant:

In a restaurant	In EduVers	Where to find it
📋 **The menu**: what you can order	**Routes**	`routes/web.php`
🚪 **The guard at the door**: "Are you allowed in?"	**Middleware**	`app/Http/Middleware/`

In a restaurant	In EduVers	Where to find it
🧑‍🍳 **The waiter**: takes your order and brings your food	**Controller**	`app/Http/Controllers/`
👨‍🍳 **The cook**: knows where all the food is and the rules for it	**Model**	`app/Models/`
🗄️ **The storage room**: where all the food is kept	**Database**	MySQL
🍽️ **The plate**: how the food looks when it reaches you	**View**	`resources/views/`
👅 **The taste tester**: checks nothing is wrong	**Tests**	`tests/`

The golden rule: every helper does only **one** job. The waiter does not cook. The plate does not decide what food is on it.

---

4. Follow one click, from start to finish

A student clicks **"My Subjects"**. Here is what happens, step by step.

Step 1: The student asks. The browser asks for the page `/student/subjects`.

Step 2: The menu is checked. (`routes/web.php`) EduVers looks at its menu and finds who should handle `/student/subjects`.

Step 3: The guard checks the student. (`app/Http/Middleware/`)

- Is this person signed in? If not, they go back to the login page.
- Is this person a **student**? If a teacher tries to open it, they get a **"403 Forbidden"** page. That means "you are not allowed here."

Step 4: The waiter takes the order. (`app/Http/Controllers/Student/SubjectController.php`) The waiter asks the cook: "Which classes does this student have?"

Step 5: The cook finds the answer. (`app/Models/User.php`) The cook knows the rule:

> A student's classes are **the classes of their section**, plus **any class the admin said yes to**.

The cook looks in the storage room (the database) and finds those classes.

Step 6: The food goes on the plate. (`resources/views/student/subjects/index.blade.php`) The page is filled in with the subject names and the teacher names.

Step 7: The page comes back. The student sees their subjects on the screen.

Every page in EduVers works with these same 7 steps.

---

5. Where is everything saved?

Everything is saved in the **database**. Think of the database as a big set of **tables**, like spreadsheets in Excel.

Table	What it keeps
`users`	Every person: admins, teachers and students
`academic_terms`	The school year and its terms
`grade_levels`	Grade 11, Grade 12, 1st Year, ...
`track_strands`	The strands and courses: ICT, ABM, HRS, ...
`sections`	The sections, like "ICT 11-A"
`subjects`	The subjects, like "Computer Programming 1"
`subject_teacher`	A **class**: one subject in one section, with its teacher
`lessons`	The lessons the teachers post
`quizzes`, `quiz_questions`	The quizzes and their questions
`quiz_submissions`	The answers and scores of the students
`class_enrollments`	Requests to join a class from another section
`announcements`	The news posted for everyone or for a section
`notifications`	The messages in the bell 🔔

Why the tables were made in a certain order

You cannot put a section in a grade level that does not exist yet. So the tables were made **one after another**, like building blocks:

1. School year (term)
2. Strands and courses
3. Grade levels
4. Sections (need a grade level **and** a strand)
5. Subjects (need a grade level and a term)
6. Classes (a subject **in** a section)
7. Lessons and quizzes (belong to a subject)

You can see this order in the folder `database/migrations`. Each file has a date in its name, so they line up from oldest to newest.

Migration or model? (People mix these up)

- A **migration** is a **building plan** for a table. It runs **one time** to build or change the table. You never change an old one. If you need a change, you write a new one.
- A **model** is the **cook** who uses the table every day and knows its rules.

---

6. How EduVers keeps everyone safe

These are good to mention in your defense.

Safety feature	What it means in simple words
Login lock	If someone types the wrong password **5 times**, they must wait before trying again. This stops people from guessing passwords.
Auto sign-out	If nobody touches the website for **30 minutes**, it signs out by itself. So nobody can use your account if you forget to sign out on a school computer.
Forgot password	You get an email with a special link. The link works **one time only**, for **60 minutes**.
Hidden passwords	Passwords are scrambled before they are saved. Not even the admin can read them.
Role check	Every page checks if you are an admin, teacher or student. You only see your own pages.
Teachers own their work	A teacher can only edit **their own** lessons and quizzes, not another teacher's.
Private files	Lesson files can only be downloaded by people who are allowed to see that lesson.
Admin says yes first	A student can't just join any class. The admin must say yes.

---

7. What tools were used to build it

Tool	What it does, in simple words
Laravel	A ready-made toolbox for building websites. It gives us the menu, the waiters and the plates.
PHP	The language the website "thinks" in.
MySQL	The database: where everything is saved.
Blade	Pages written in HTML with little "holes" where data goes.

Tool	What it does, in simple words
Tailwind CSS	Gives the website its colors, sizes and spacing.
Vite	Packs the colors and buttons together so the pages load fast.
PHPUnit	The taste tester. It checks that everything still works.
Laragon	A program on the computer that runs PHP and MySQL in one click.

Online version: the website runs on **Railway** (the computer on the internet that shows the website), and the database is on **TiDB Cloud** (a database on the internet). Emails are sent through **Brevo**.

The login page was **written by hand**, not copied from a ready-made kit. That is why it looks like the school, and why we can explain every rule in it.

---

8. The folders, one line each

```
eduvers/
├── routes/web.php        the menu: every page address of the website
├── app/
│   ├── Http/Controllers/ the waiters (one folder each for Admin, Teacher, Student)
│   ├── Models/           the cooks (User, Subject, Quiz, …)
│   ├── Http/Middleware/  the guards at the door
│   ├── Http/Requests/    checks that forms are filled in correctly
│   ├── Support/          helpers, like the Gradebook
│   └── Enums/            short fixed lists, like the roles
├── resources/
│   ├── views/            the plates: the pages people see
│   ├── js/app.js         small moving parts: pop-ups, the bell, the clock
│   └── css/app.css       colors and spacing
├── database/
│   ├── migrations/       the building plans of the tables
│   └── seeders/          sample data and demo accounts
├── tests/                the taste tester
├── public/               the only folder the internet can see (pictures, logo)
├── config/               settings
└── handover/             files to install and run EduVers (this file is here)
```

---

9. How to open and run it

For the defense (show only): Double-click **"Start EduVers"** on the Desktop. Keep the black window open. The website opens at `http://127.0.0.1:8000`.

To change the code: Double-click **"Edit EduVers"** on the Desktop. It opens the code in VS Code. When you are done, press any key in its black window so your changes are saved.

To check that everything still works, open Laragon's Terminal and type:

```
php artisan test
```

Green means everything works. Red means something broke.

---

10. How a new feature is added

Every feature in EduVers was made with the same 6 steps:

1. **Building plan:** make a migration if a new table or column is needed.
2. **Cook:** make or change the model, and write its rules there.
3. **Menu:** add the page address in `routes/web.php`.
4. **Waiter:** make the controller that gets and saves the data.
5. **Plate:** make the page in `resources/views/`.
6. **Taste test:** write a test that fails if the feature breaks.

Then run `php artisan test`. If it is all green, the feature is done.

---

11. When something goes wrong

What you see	What it usually means	What to do
404 Not Found	The page address is not on the menu	Check `routes/web.php`
403 Forbidden	You are signed in as the wrong person	Sign in with the right account
500 Server Error	Something broke in the code	Read `storage/logs/laravel.log` (the newest error is at the bottom)
The page has no colors	The styles were not built	Run `npm run build`

What you see	What it usually means	What to do
"Column not found"	A building plan was not run yet	Run `php artisan migrate`
"Connection refused"	The database is not running	Open Laragon and click **Start All**

---

12. Questions the panel may ask (with simple answers)

"What is EduVers?" A website for our school where teachers post lessons and quizzes, and students learn and take quizzes online. It has three users: admin, teacher and student.

"How does a page work?" Like a restaurant. The menu (route) finds who handles the request. The guard (middleware) checks if you are allowed. The waiter (controller) asks the cook (model). The cook gets the data from the storage room (database). The data goes on a plate (view) and is sent back to you.

"Why did you use Laravel?" It gives us safe, tested parts for things every website needs: logins, pages, the database and emails. So we could spend our time on the school's needs instead of building everything from zero.

"How are quizzes checked?" The right answer is saved with each question. When the student submits, EduVers compares each answer with the right one and shows the score **right away**. A student can only take each quiz **once**.

"How do you keep the data safe?" Passwords are scrambled. Every page checks your role. Wrong passwords lock the login after 5 tries. The website signs out by itself after 30 minutes of doing nothing.

"What happens when a teacher posts a lesson?" The lesson is saved. Then every student in that class gets a message in the bell 🔔 and an email.

"How do you know it works?" We wrote tests. One command, `php artisan test`, checks the whole system in a few seconds and shows green if everything works.

"Can a student join a class from another section?" Yes, but only if the admin says yes. The student or the teacher sends a request, and the admin approves or rejects it.

---

13. Words to remember

Word	Simple meaning
Website / web app	Pages you open in a browser like Chrome

Word	Simple meaning
Database	Where everything is saved, like big Excel tables
Table	One list in the database, like a list of students
Route	A page address and who handles it (the menu)
Middleware	A check that runs before the page opens (the guard)
Controller	The code that handles one page (the waiter)
Model	One kind of thing, like User or Quiz, and its rules (the cook)
View / Blade	The page people see (the plate)
Migration	A building plan that makes or changes a table
Seeder	Fills a new database with sample data
Test	A check that the code still works (the taste tester)
Laravel	The toolbox used to build EduVers
Artisan	The command helper: `php artisan …`
Server	The computer that runs the website
Deploy	Putting the website on the internet